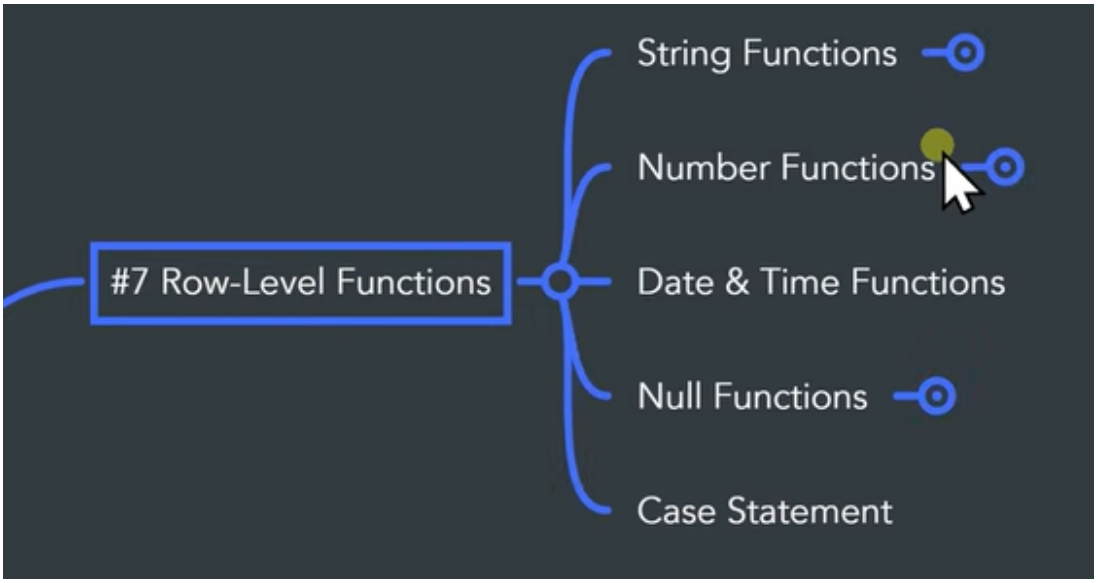
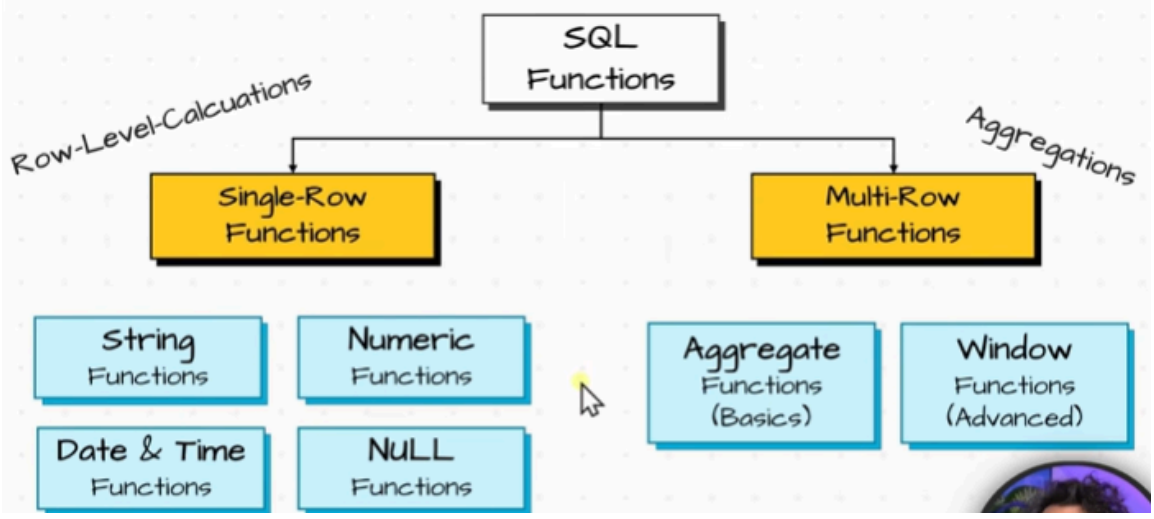
Row-Level Functions

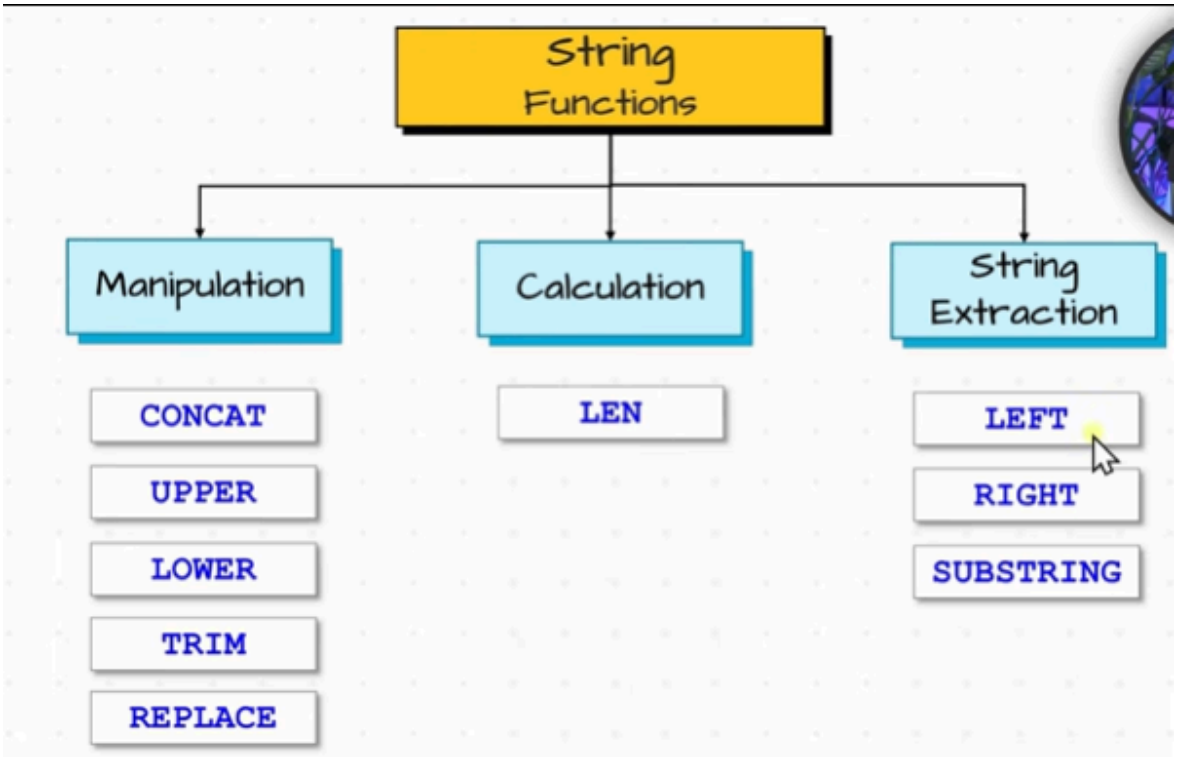


1. Nested Functions are very widely used in ROW-LEVEL Functions



1. String Functions

1. to modify the String data in the tables.
2. Like LOWER() ,UPPER(),LEN() etc..



1. SELECT f_name,country , CONCAT(f_name,l_name) AS full_name from customers ⇒ concatinating strings
2. SELECT f_name,country , LOWER(f_name) AS full_name from customers ⇒ Lower strings
3. For Trim → to remove leading&&ending spaces

```
-- Find customers whose first name contains leading or trailing spaces

SELECT
    first_name
FROM customers
WHERE first_name != TRIM(first_name)
```

4. SELECT '123-456-7890' AS Phone Replace('123-456-7890','-','') as clean

   Here in REPLACE function first we need to give the column name and then what charcter to be replaced , then to which char the old character to replace

5. LEN → to calcuate the length of the string

LEN(f_name)-LEN(TRIM(f_name)) >0 → nested operations ⇒ to know which have leading/ending spaces

6. LEFT,RIGHT takes two parameters → 1. column name, 2. how many characters needed to take

```
SELECT
    first_name,
    LEFT(TRIM(first_name), 2) first_2_char,
    RIGHT(first_name, 2) last_2_char
FROM customers
```

Results | Messages

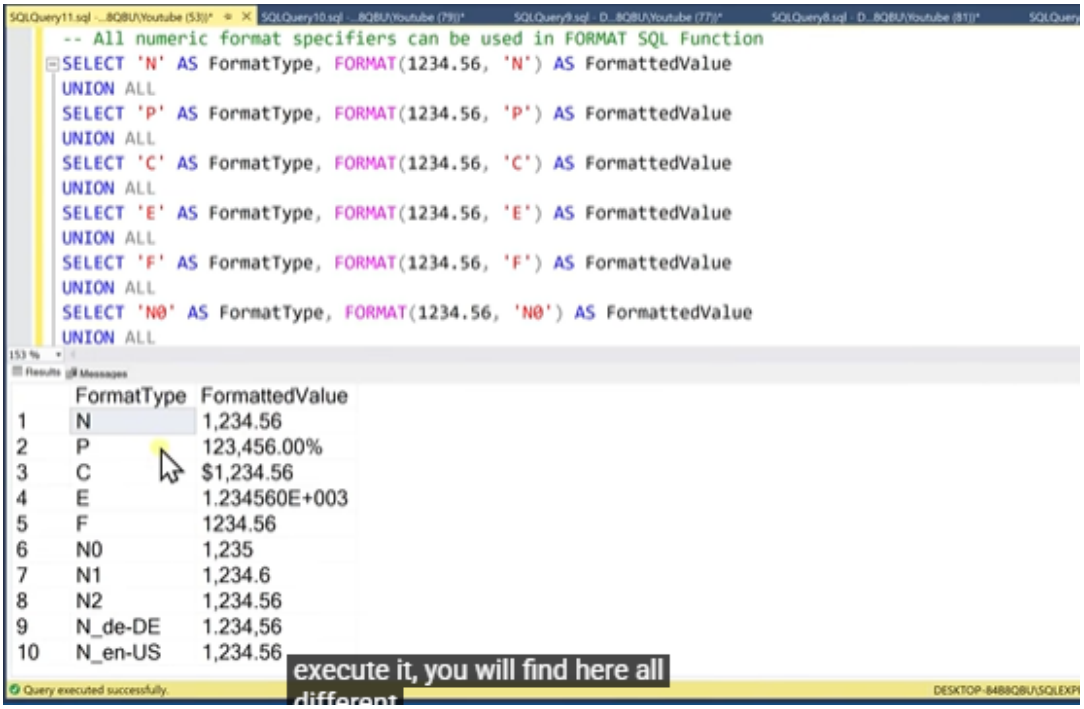
	first_name	first_2_char	last_2_char
1	Maria	Ma	ia
2	John	Jo	hn
3	Georg	Ge	rg
4	Martin	Ma	in
5	Peter	Pe	er

7. SUBSTRING → takes 3 parameters
    a. column name , Start index, Length of the substring

SUBSTRING(TRIM(f_name),2,LEN(f_name)) as substring_from_index_2_to_end

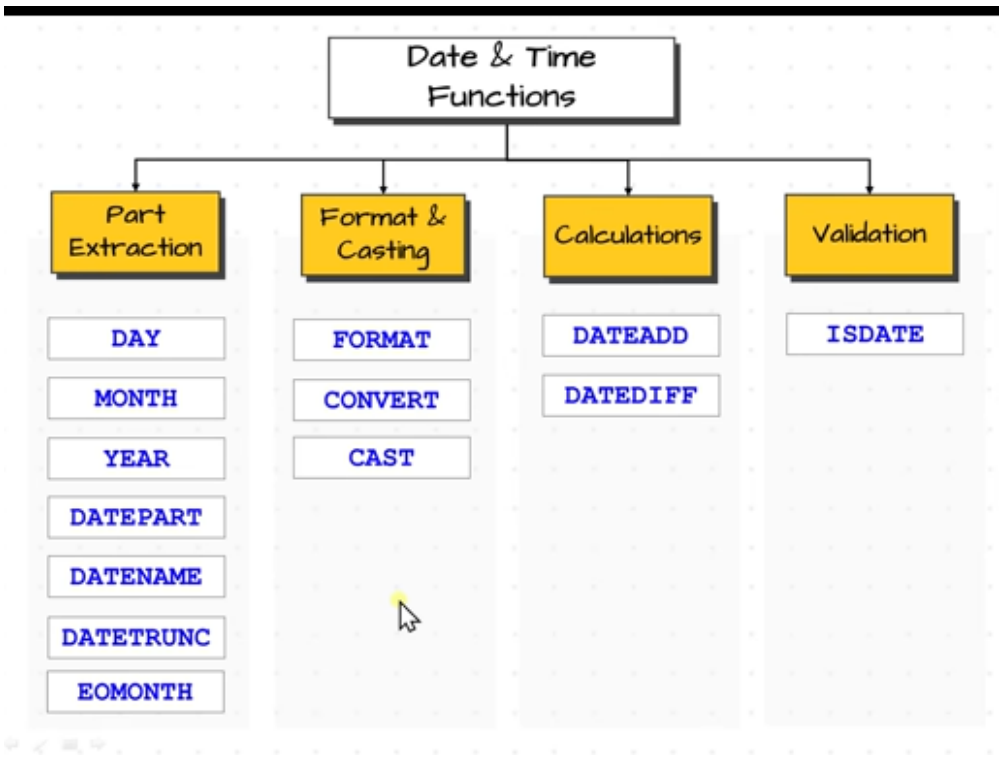
2. Number Functions

1. Round → takes two parameters
    a. column_name , upto how many decimal position the round off need to be
    b. ROUND(height,2) → after decimal we have two
    c. ABS(column_name) ⇒ to convert all values to positive



3. Date & Time Functions

1. To get date and time we have three methods →
    a. from the table column CreationTime
    b. Hardcoded constant string value
    c. ** GETDATE() ⇒ returns the current date and time at the moment when the query is executed
2. Functions



1. Part Extraction

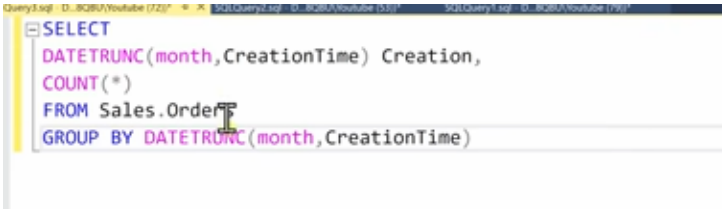
1. DAY() , MONTH(), YEAR() → returns their repective values from the date
    a. These gives in the form of numbers → like month(1,2,....12)
2. DatePart() → returns a specific part of date as a number

    DATEPART(hour,column_name) ⇒ returns the hour part,

    make sure that column must be date_time
3. DateName() → returns the string formate answer
    a. DATENAME(month,creationtime→column name) ⇒ gives january,february,march..dec
    b. DATENAME(weekday,creationtime→column name) ⇒ mon,tues...sun
    c. *** DAY,YEAR (numeric) ⇒ will be numeric only as there are no strings
4. DATETRUNC() → Truncates the date to specific part



a. here it takes the part,column_name as arguements
b. DATETRUNC(month,creationtime) ⇒ after month all the formates will be reset → months,day to 01. hours,minutes,seconds to 00.



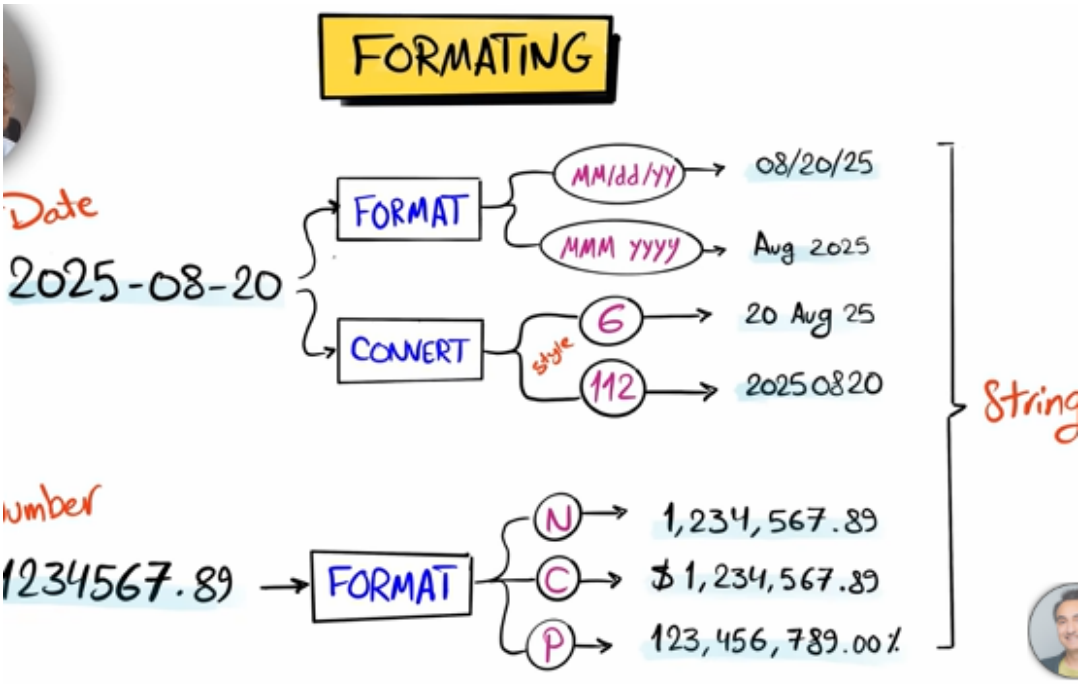
```
SELECT
DATETRUNC(month,CreationTime) Creation,
COUNT(*)
FROM Sales.Orders
GROUP BY DATETRUNC(month,CreationTime)
```

this gives the orders from that particular month

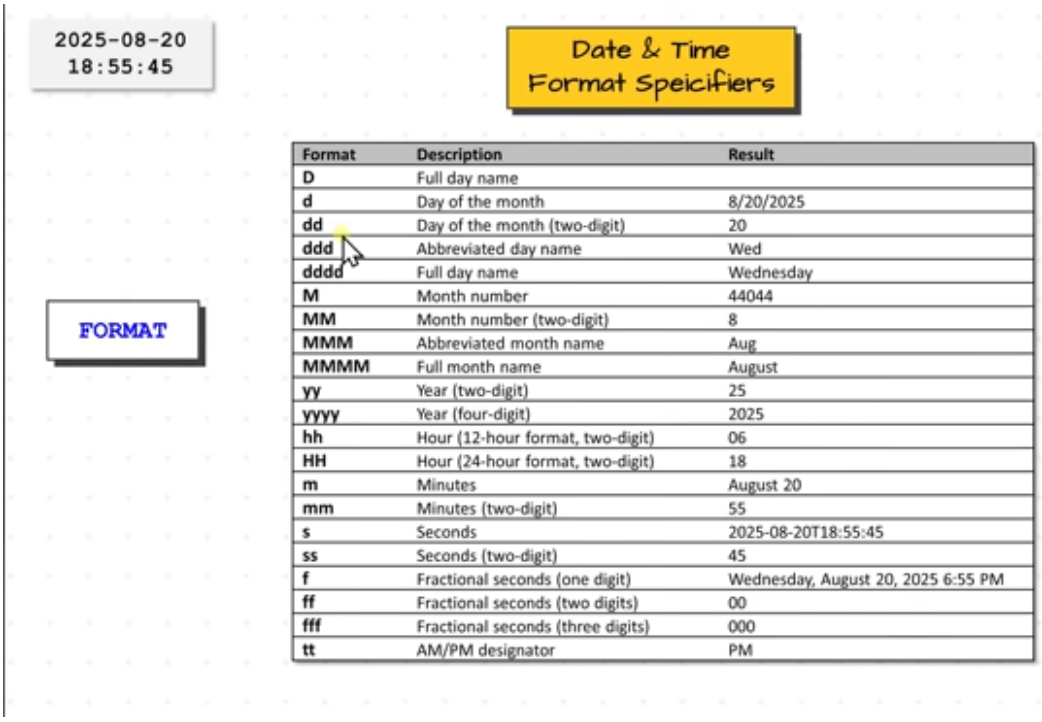
5. EOMONTH ⇒ end of month
   a. returns the date to its end of the month

2. Formate and Casting

1. Changing the format of a value from one to another



2. Casting means changing the data type from one to another



Format	Description	Result
D	Full day name	
d	Day of the month	8/20/2025
dd	Day of the month (two-digit)	20
ddd	Abbreviated day name	Wed
dddd	Full day name	Wednesday
M	Month number	44044
MM	Month number (two-digit)	8
MMM	Abbreviated month name	Aug
MMMM	Full month name	August
yy	Year (two-digit)	25
yyyy	Year (four-digit)	2025
hh	Hour (12-hour format, two-digit)	06
HH	Hour (24-hour format, two-digit)	18
m	Minutes	August 20
mm	Minutes (two-digit)	55
s	Seconds	2025-08-20T18:55:45
ss	Seconds (two-digit)	45
f	Fractional seconds (one digit)	Wednesday, August 20, 2025 6:55 PM
ff	Fractional seconds (two digits)	00
fff	Fractional seconds (three digits)	000
tt	AM/PM designator	PM

a. formate

1. takes the column_name ,formate as the parameters

Syntax

```
FORMAT(value, format [,culture])
```

Optional

Examples

```
FORMAT(OrderDate,'dd/MM/yyyy')
```

```
FORMAT(OrderDate,'dd/MM/yyyy','ja-JP')
```

```
FORMAT(1234.56, 'D' ,'fr-FR')
```

Default Cultuer = 'en-US'

```
SELECT
OrderID,
CreationTime,
FORMAT(CreationTime, 'MM-dd-yyyy') USA_Format,
FORMAT(CreationTime, 'dd') dd,
FORMAT(CreationTime, 'ddd') ddd,
FORMAT(CreationTime, 'dddd') dddd,
FORMAT(CreationTime, 'MM') MM,
FORMAT(CreationTime, 'MMM') MMM,
FORMAT(CreationTime, 'MMMM') MMMM
FROM Sales.Orders
```



OrderID	CreationTime	USA_Format	dd	ddd	dddd	MM	MMM	MMMM
1	2025-01-01 12:34:56.0000000	01-01-2025	01	Wed	Wednesday	01	Jan	January
2	2025-01-05 23:22:04.0000000	01-05-2025	05	Sun	Sunday	01	Jan	January
3	2025-01-10 18:24:08.0000000	01-10-2025	10	Fri	Friday	01	Jan	January
4	2025-01-20 05:50:33.0000000	01-20-2025	20	Mon	Monday	01	Jan	January
5	2025-02-01 14:02:41.0000000	02-01-2025	01	Sat	Saturday	02	Feb	February

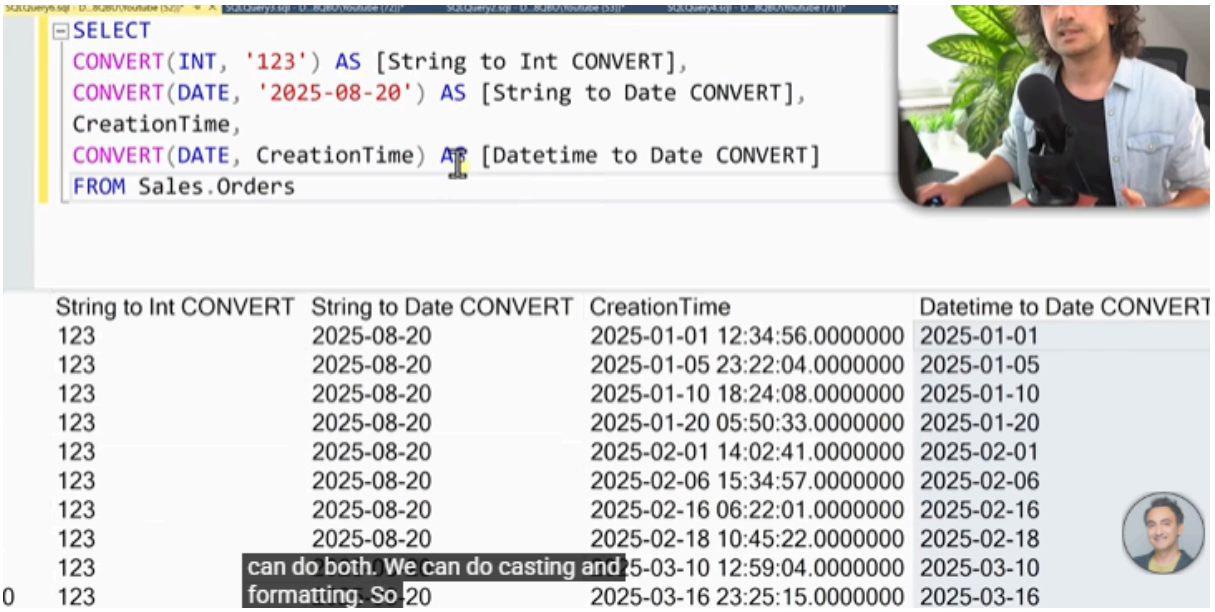
b. Formate + GroupBy

```
SELECT
FORMAT(OrderDate, 'MMM yy') OrderDate,
COUNT(*)
FROM Sales.Orders
GROUP BY FORMAT(OrderDate, 'MMM yy')
```

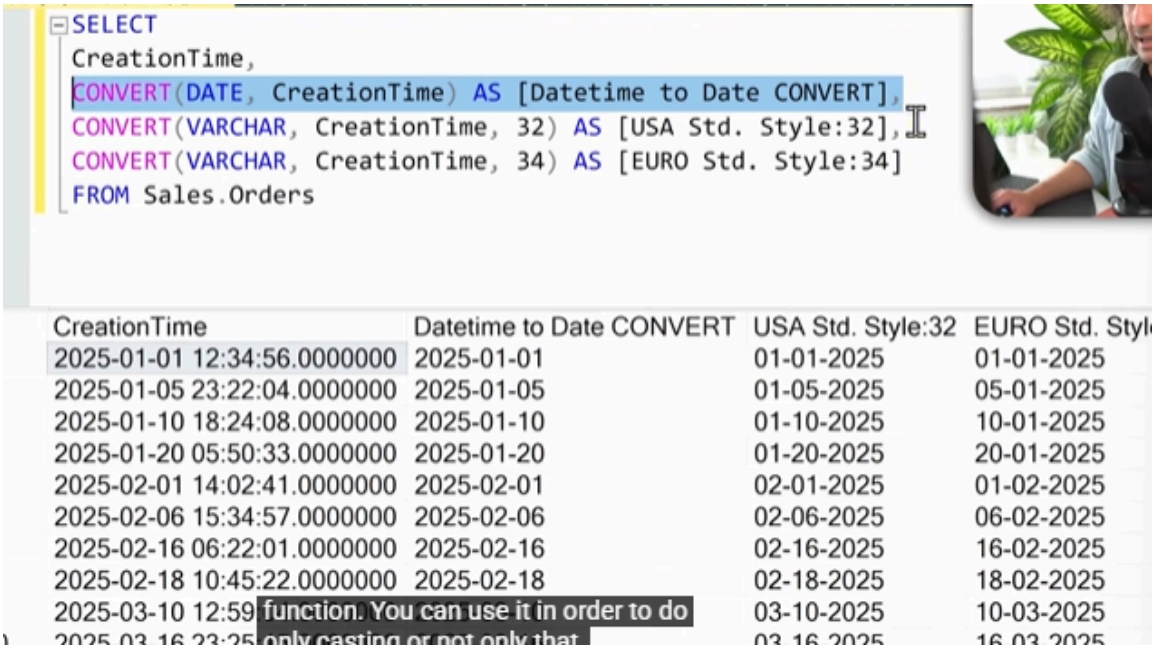
OrderDate	(No column name)
Feb 25	4
Jan 25	4
Mar 25	2

c. Convert

1. Convers a date or time value to a dfferent data type
2. data_type to which it should be converted ,column_name
    a. Convert(data_type,column_name,style['optional'])

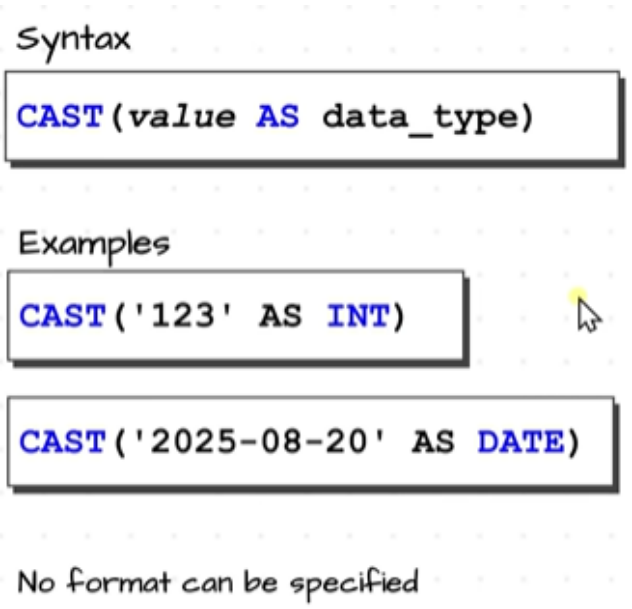


3. style⇒ 32 means USA(month-date-year) / 34 means EURO (date-month-year)

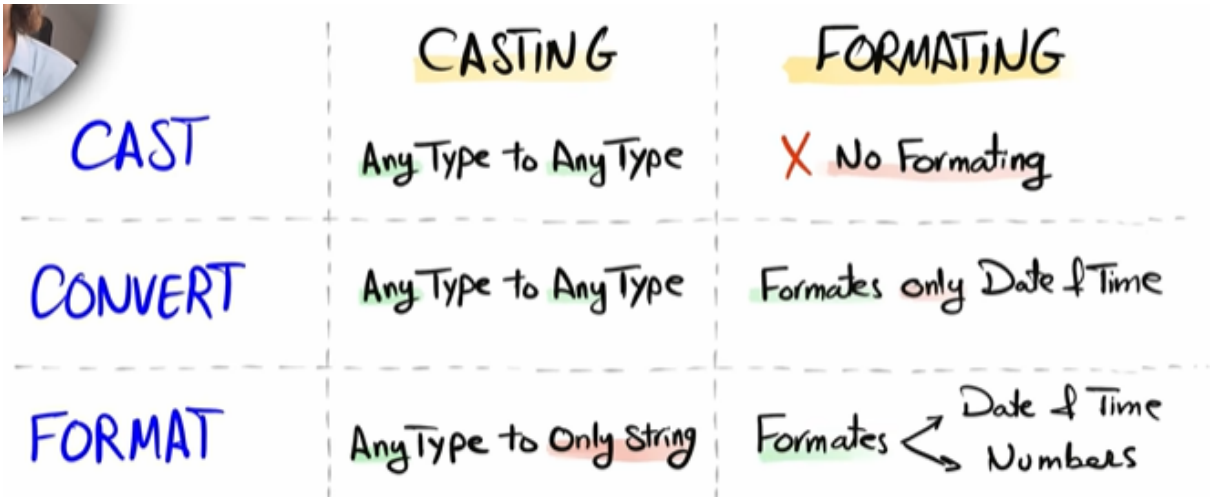


d. Cast

1. COnverts a value to a specified data type



e. Difference between Cast, Convert, Format



3. Calculations

1. DATEADD() → to add a specific time interval to the date
    a. Takes three paramters → part(like month,day,year) , interval(jow much we need to add to the current date),column name

Syntax

```
DATEADD(part, interval, date)
```

Examples

```
DATEADD(year, 2, OrderDate)
```

```
DATEADD(month, -4, OrderDate)
```

2. DATEDIFF() → to take the difference between to date's

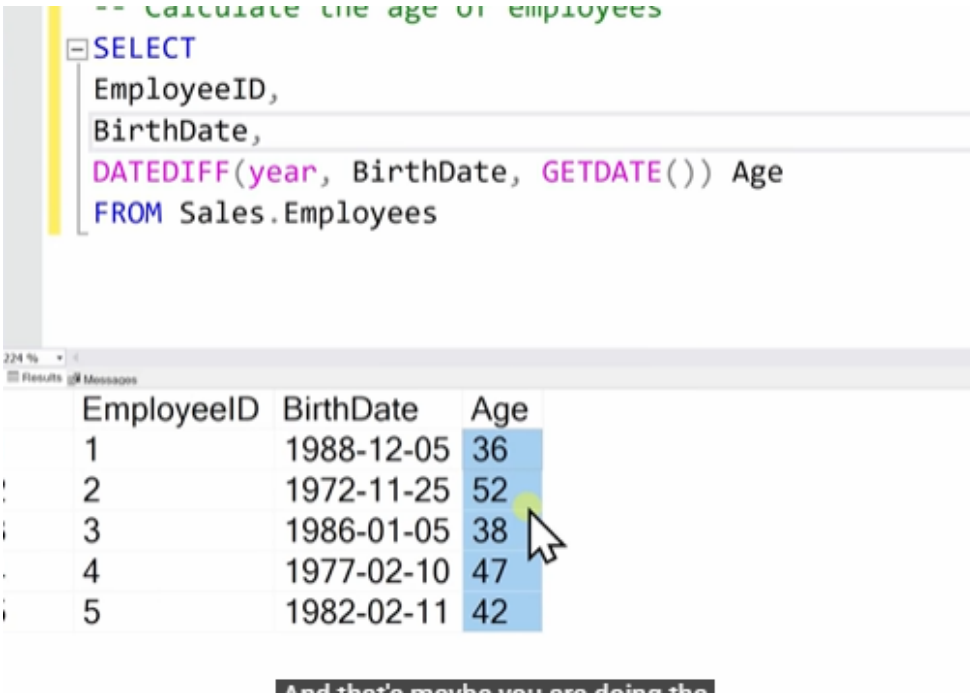
Syntax

```
DATEDIFF(part, start_date, end_date)
```

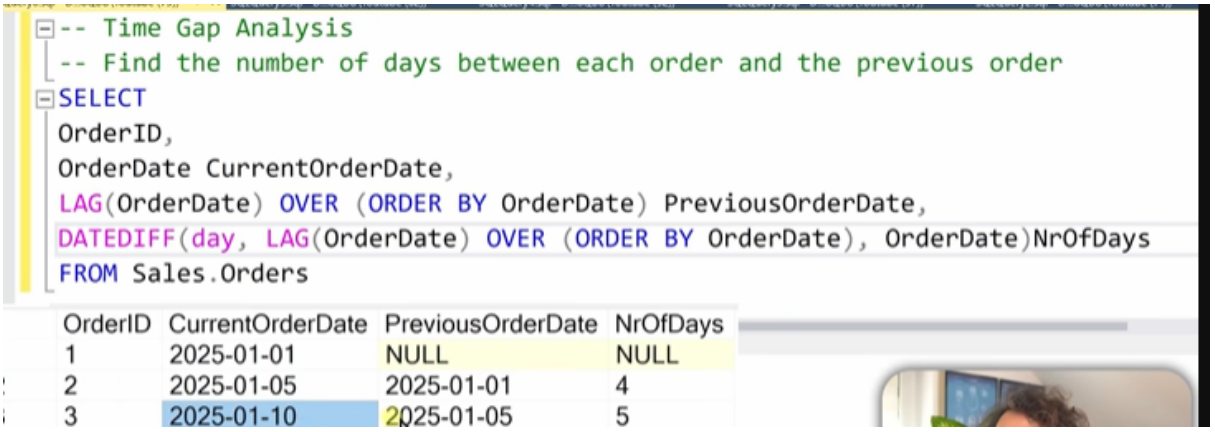
Examples

```
DATEDIFF(year, OrderDate, ShipDate)
```

```
DATEDIFF(day, OrderDate, ShipDate)
```



LAG + ORDER-BY + DATE



a. actually LAG is a Window function , which is used over the Aggregation function

b. first order-by makes the orderdate in ascending order , so then lag makes by default neglect one item , so we get the second item from ordered data → which is previous order

4. Validation functions

1. Check if a value is a date
2. returns 1 if the string value is a valid date

3. Standard formate that ISDATE function checks whether it is a date or not is → year-month-day

   or it can considers a single year as date to return 1

   year → 1, month → 0, day → 0 only single year is considered as date

ISDATE + CAST + CASE + SUBQUERY functions

```
SELECT
    --CAST(OrderDate AS DATE) OrderDate,
    OrderDate,
    ISDATE(OrderDate),
    CASE WHEN ISDATE(OrderDate) = 1 THEN CAST(OrderDate AS DATE)
    END NewOrderDate
FROM
(
    SELECT '2025-08-20' AS OrderDate UNION
    SELECT '2025-08-21' UNION
    SELECT '2025-08-23' UNION
    SELECT '2025-08'
)t
```

	OrderDate	(No column name)	NewOrderDate
1	2025-08	0	NULL
2	2025-08-20	1	2025-08-20
3	2025-08-21	1	2025-08-21
4	2025-08-23	1	2025-08-23

1. here sub-query has 4rows and column name OrderDate . for sub-query we need to give alias (here gave as t)
2. then if CASE function is not used and directly used the CAST(OrderDate) then it gives error , because 2025-08 is not a date
3. so to remove error and to make not a date function to null we use this
4. so ISDATE made 1,0's if it was a date
5. so using that and CASE WHEN function we made casting , else NewOrderDate → null

Week

- **MySQL**: You can use the `WEEK()` function, which returns the week number of a given date (a number from 0 to 53). It also supports an optional mode parameter to define the first day of the week.Other MySQL functions

include `WEEKOFYEAR()` (returns the week number from 1 to 53, assuming Monday as the first day) and `YEARWEEK()` (returns the year and week number combined).

sql

```
SELECT WEEK('2024-03-03');
```

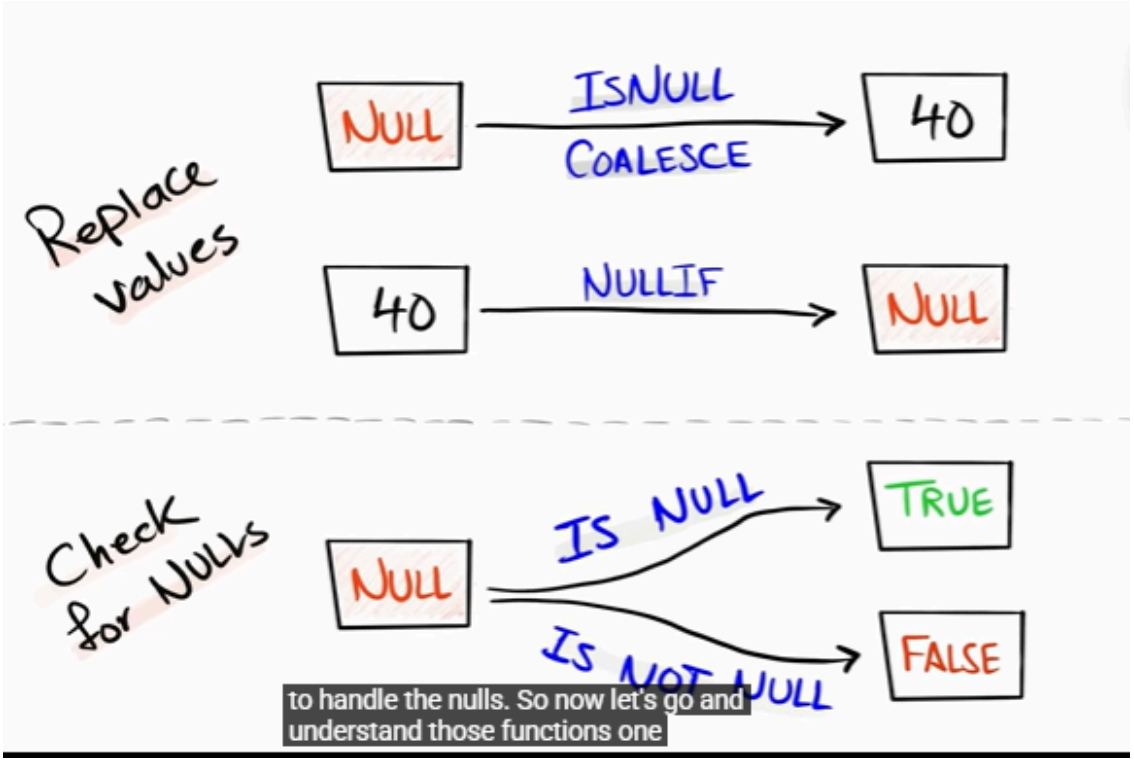
- **SQL Server**: You use the `DATEPART()` function with the `WEEK`, `WK`, `WW`, or `ISO_WEEK` datepart arguments.Note that the `DATEPART(WEEK, ...)` result can be affected by the `SET DATEFIRST` session setting, which determines the first day of the week. The `ISO_WEEK` argument is not affected by `SET DATEFIRST` and always follows the ISO 8601 standard.

  sql

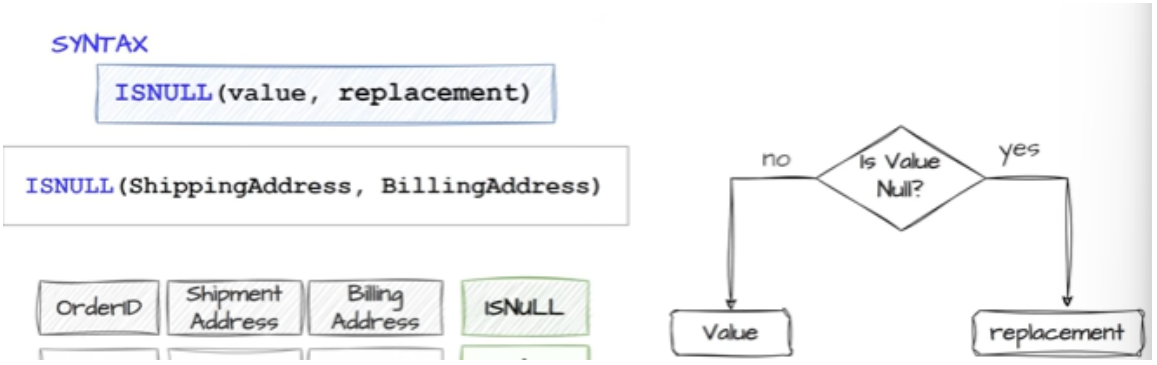
  ```
  SELECT DATEPART(WEEK, GETDATE());
  -- or for ISO 8601 standard week numbering (Monday as first day):
  SELECT DATEPART(ISO_WEEK, GETDATE());
  ```

4. Null Functions

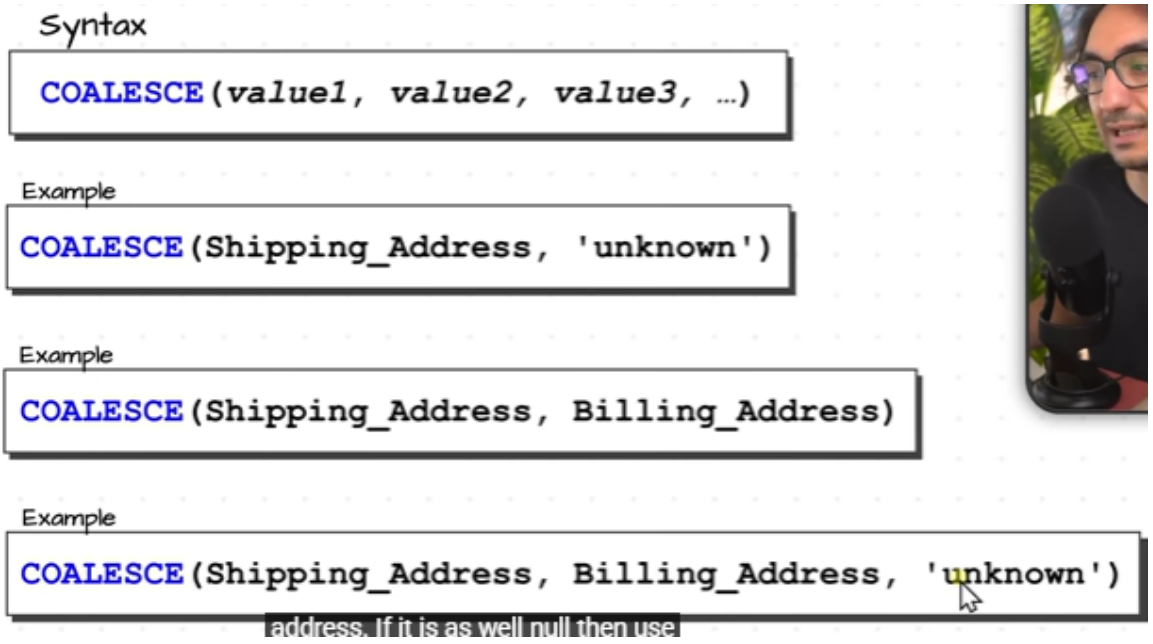
1. Null ⇒ it means nothing,unknown,
   a. NOT a ZERO, Not Empty STRING, Not a Blank space



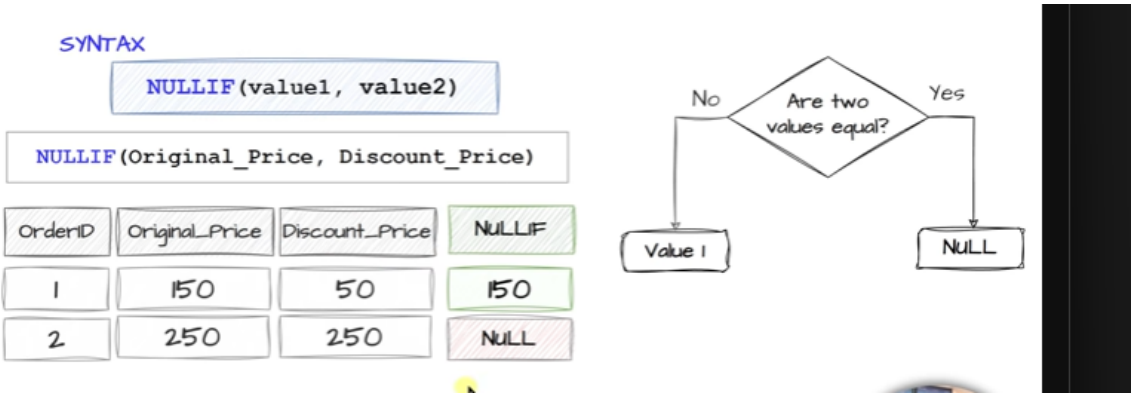
1. ISNULL → if a value is null then this converts it to a specified value. else takes the correct value
    a. Takes Arguements column_name to check , replacement_value/replacement_column

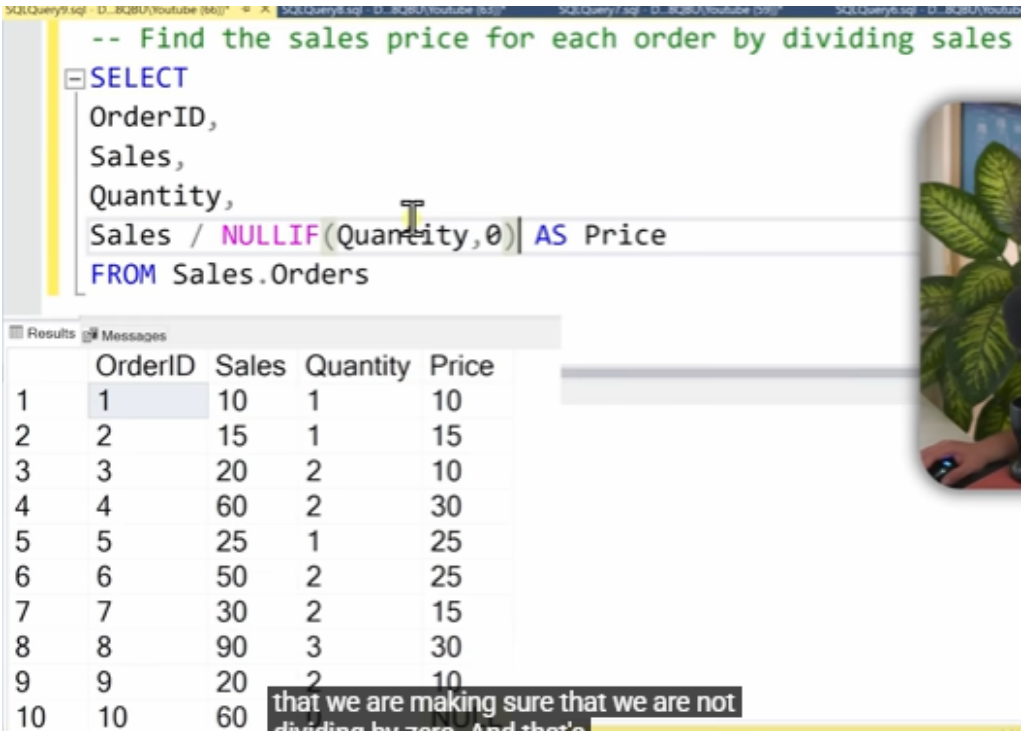


2. COALESCE → Returns the first non-null value from a list



3. ISNULL is faster than COALESCE
4. NULLIF → to check two values arre equal or not
    a. if equal then returns null
    b. else returns the first value
    c. It takes only two values as parameters





5. IS NULL → returns true if value is null, else false
6. IS NOT NULL → returns true is not null, else false

a. NULL + Aggregation function

1. SUM, etc → takes the values of that particular row
    a. so if a null is there then they ignore it , as they consider only values
2. But there are sum functions like COUNT,AVG → they consider the rows to count
    a. if a null is there then it ignores it, but we need to count it , so NULL FUNCTIONS help there

```
SELECT
CustomerID,
Score,
COALESCE(Score,0) Score2,
AVG(Score) OVER () AvgScores,
AVG(COALESCE(Score,0)) OVER() AvgScores2
FROM Sales.Customers
```

	CustomerID	Score	Score2	AvgScores	AvgScores2
1	1	350	350	625	500
2	2	900	900	625	500
3	3	750	750	625	500
4	4	500	500	625	500
5	5	NULL	0	625	500

3. if null → that row is neglected to divided by 4, if making it to '0' then total rows are 5 so divided by 5

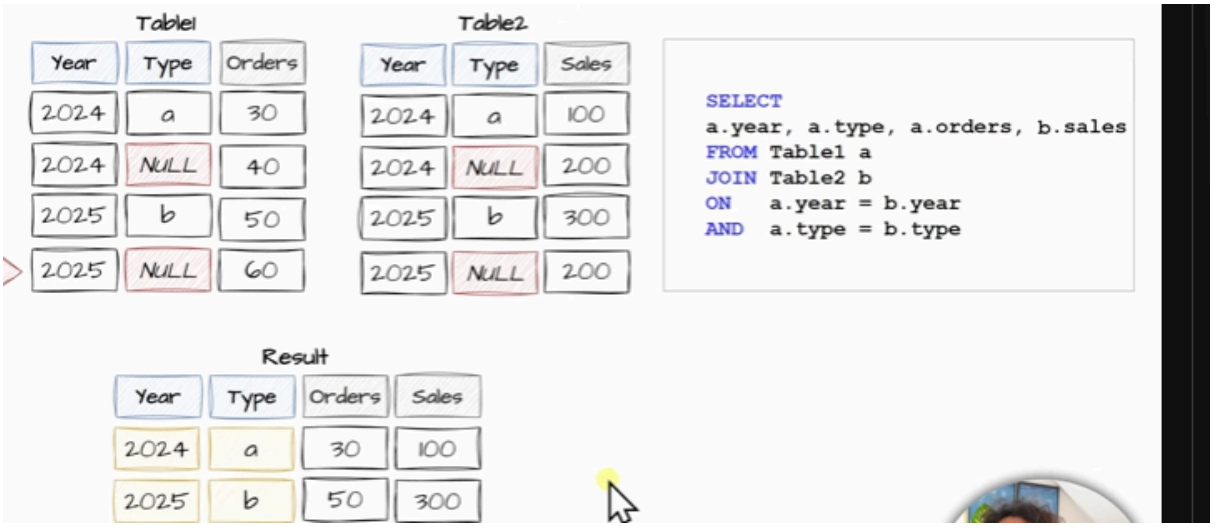
b. NULL + OPERATORS



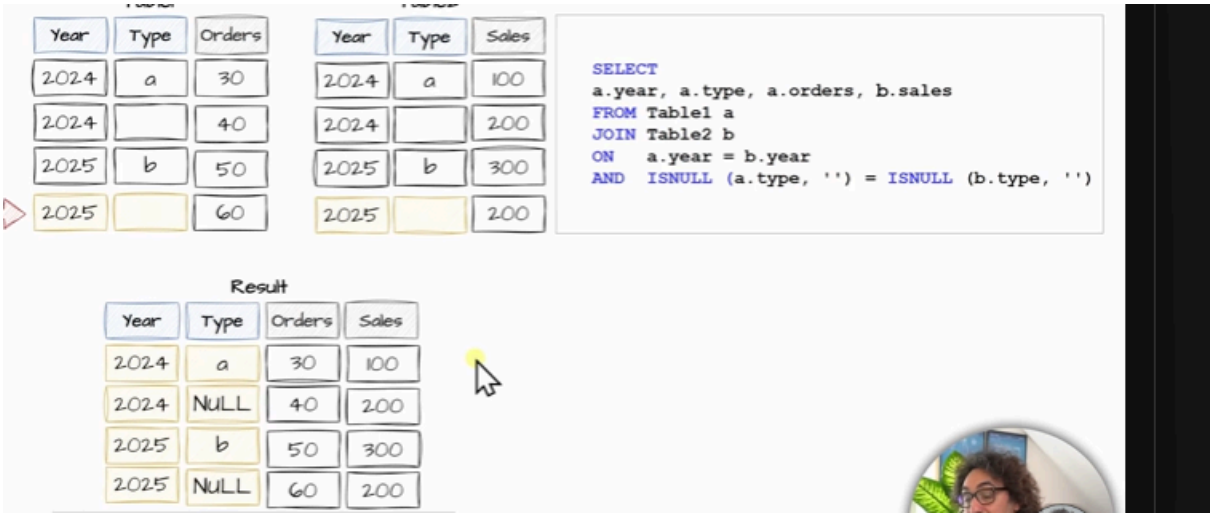
1. null+any character → null, null+value → null
2. so for any arithemetic operators , we need to replace null value
3. so there COALESCE helps us

c. NULL + JOIN

1. null=null , is considers in valid in joining



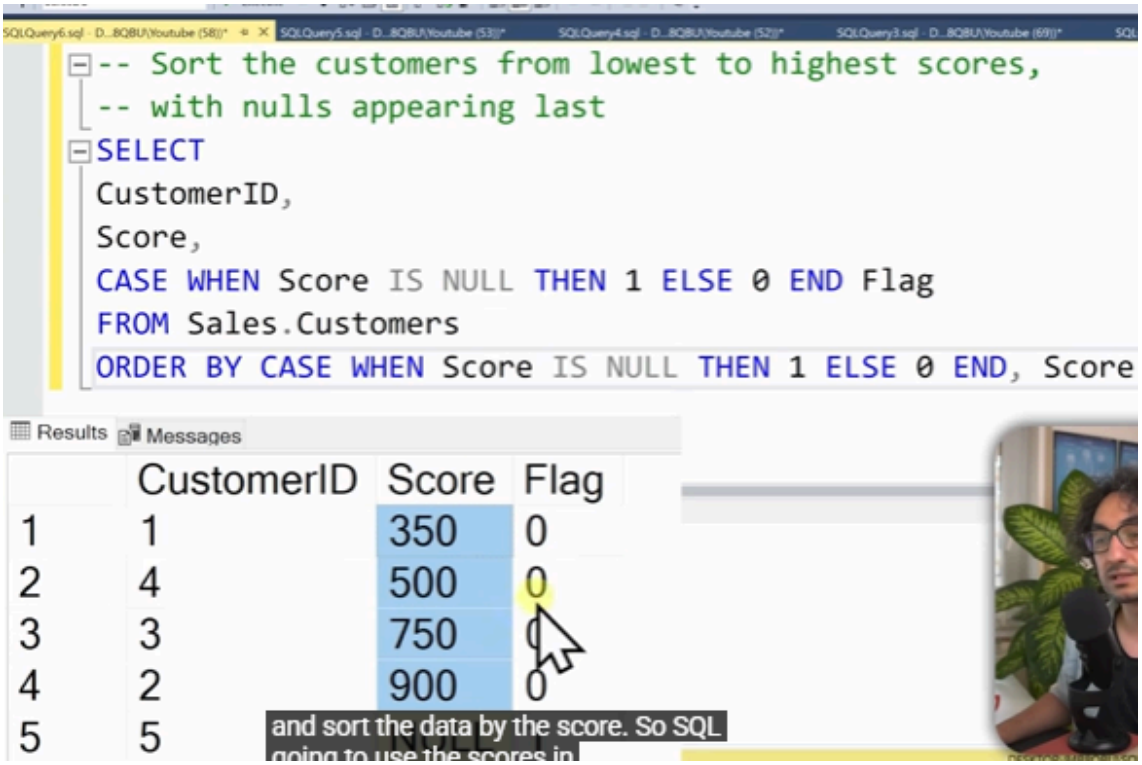
a. without ISNULL/COALESCE



a. with handling NULL's

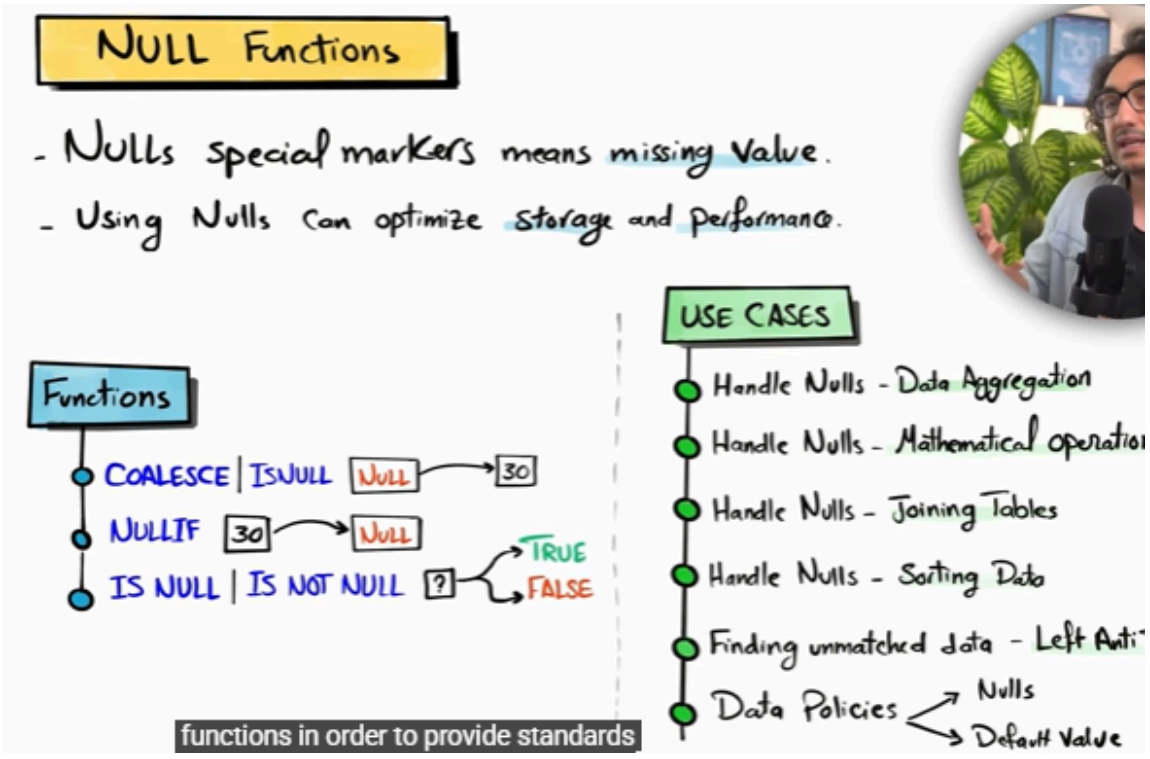
d. NULL + SORTING

1. actually null takes the lowest value among all the values in that row
2. But to sort the data , in such a way that null takes the highest value

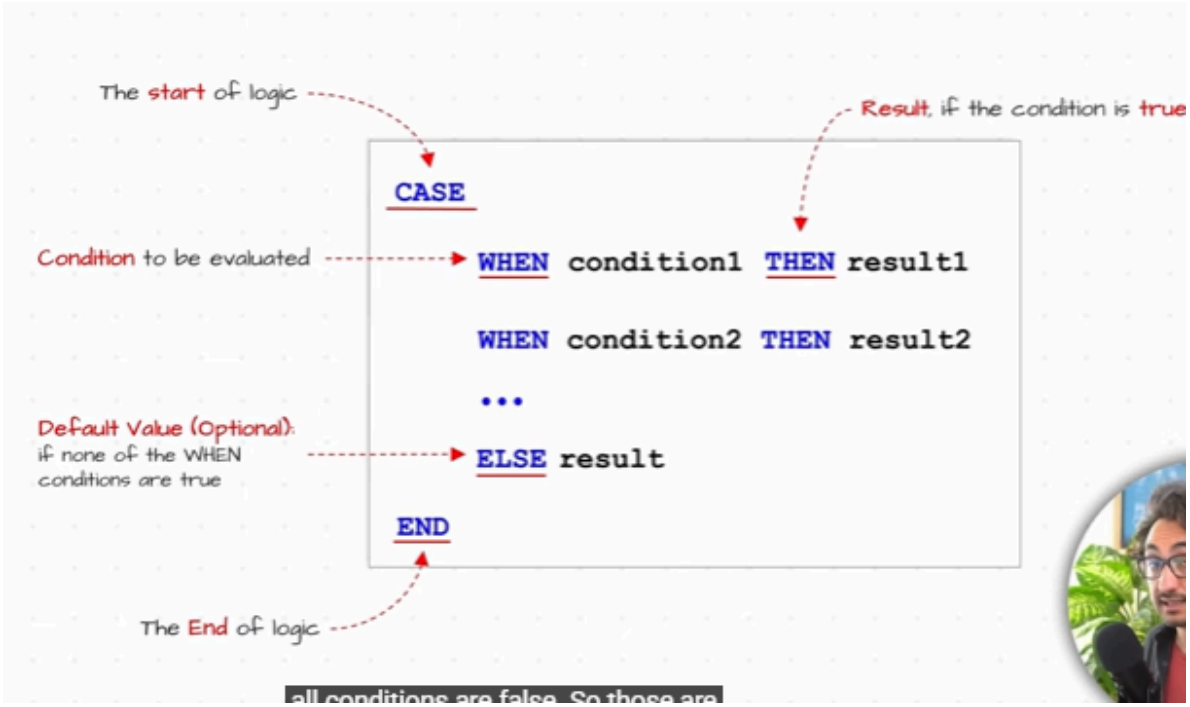


c. here asceding order only , but the null is at last

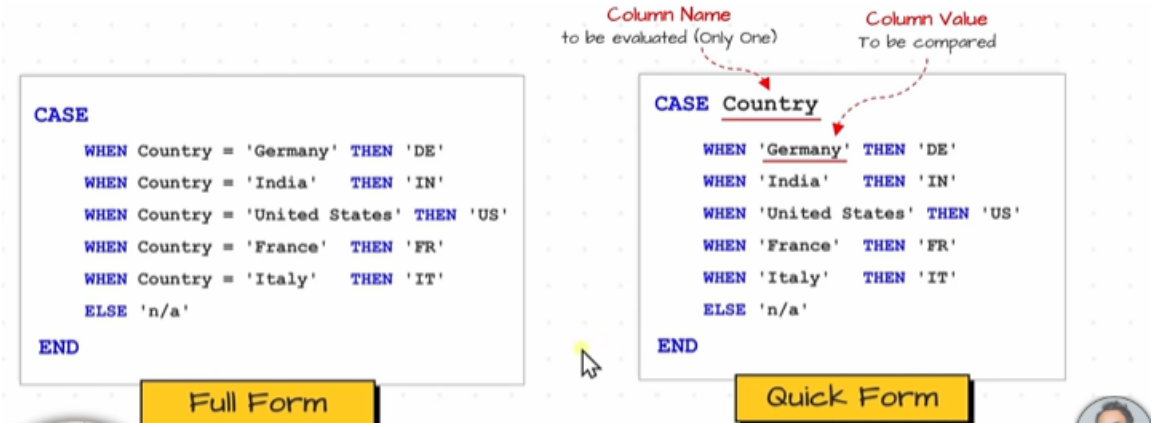
d. Order by works here in such a way that first it will sort according to flag→0,1's so null goes to last. Then it sorts on basis of Score



5. Case Statements



1. If we not write ELSE then by default, and ,when conditions not satisfied then it returns NULL else it returns Then results
2. those result1,result2,result3..... must be of same data-type
3. If conditions are same then we can write like



4. Main Purpose of case statements
    a. Data Transformation → to create a new column bases on existing data(example shown in 4th point)

b. Categorizing Data → Group the data into different categories based on certain conditions (example shown in 4th point)

c. Mapping → Transform the values from one form to another

```
-- Retrive employee details with gender displayed as full text
SELECT
EmployeeID,
FirstName,
LastName,
Gender,
CASE
    WHEN Gender = 'F' THEN 'Female'
    WHEN Gender = 'M' THEN 'Male'
    ELSE 'Not Avaiable'
END GenderFullText
FROM Sales.Employees
```

5. MAIN ADVANTAGE

a. Case statements can be written anywhere , there is no specific order for it

```
   Sort the categories from highest sales to lowest */
SELECT
Category,
SUM(Sales) AS TotalSales
FROM(
    SELECT
    OrderID,
    Sales,
    CASE
        WHEN Sales > 50 THEN 'High'
        WHEN Sales > 20 THEN 'Medium'
        ELSE 'Low'
    END Category
    FROM Sales.Orders
)t
GROUP BY Category
ORDER BY TotalSales DESC
```

	Category	TotalSales
1	High	210
2	Medium	105
3	Low	65

b. Handling Nulls → replacing NULLs with specific vaues

```sql
SELECT
CustomerID,
LastName,
Score,
CASE
    WHEN Score IS NULL THEN 0
    ELSE Score
END ScoreClean,

AVG(CASE
        WHEN Score IS NULL THEN 0
        ELSE Score
    END) OVER () AvgCustomerClean,

AVG(Score) OVER() AvgCustomer
FROM Sales.Customers
```

c. with aggregation functions

```sql
-- Count how many times each customer has made an order with sales greater than 30
SELECT
    CustomerID,
    SUM(CASE
        WHEN Sales > 30 THEN 1
        ELSE 0
    END) TotalOrdersHighSales,
    COUNT(*) TotalOrders
FROM Sales.Orders
GROUP BY CustomerID
```

6. After END we can write the column_name to create a new column → so that said the results should be of same column